base.h

```
#pragma once

#include <bits/stdc++.h>
#include <acm/TypeDef.hpp>
template <class T>
constexpr auto limit_min = std::numeric_limits<int>::min();
template <class T>
constexpr auto limit_max = std::numeric_limits<int>::max();
std::array<char, 1 << 10 << 10 << 3> cin_buffer{};
std::array<char, 1 << 10 << 10 << 3> cout_buffer{};
void solve();
void preProcess() {
    std::cin.tie(nullptr)->sync_with_stdio(false);
    std::cin.rdbuf()->pubsetbuf(cin_buffer.data(), cin_buffer.size());
    std::cout.rdbuf()->pubsetbuf(cout_buffer.data(), cout_buffer.size());
}
#define SINGLE_SOLVE int main() {\
    preProcess();\
    solve();\
}
#define MULTI_SOLVE int main() {\
    preProcess();\
    int t = 1;\
    std::cin >> t;\
    while (t--) solve();\
}
```

acm

CompGeo

ConvexHull.hpp

```
#pragma once
#include "../TypeDef.h"

struct Vector2D
{
    i64 x;
    i64 y;

    Vector2D() = default;
    Vector2D(i64 x_, i64 y_) : x(x_), y(y_) {}
    Vector2D(const Vector2D&) = default;
    Vector2D(Vector2D&&) = default;

    Vector2D& operator=(const Vector2D&) = default;
    Vector2D& operator=(Vector2D&&) = default;
```

```
    i128 operator*(const Vector2D& vec) const {
        return 1ll * x * vec.x + y * vec.y;
    }

    friend i128 cross_multi(const Vector2D& a, const Vector2D& b) {
        return 1ll * a.x * b.y - b.x * a.y;
    }

    double len() const {
        return std::sqrt(i64(x) * x + i64(y) * y);
    }
};

struct Point {
    i64 x;
    i64 y;
    Point() = default;
    Point(i64 x_, i64 y_) : x(x_), y(y_) {}
    Point(const Point&) = default;
    Point(Point&&) = default;

    Point& operator=(const Point&) = default;
    Point& operator=(Point&&) = default;

    Vector2D operator-(const Point& p) const {
        return Vector2D{x - p.x, y - p.y};
    }

    auto operator<=>(const Point&) const = default;
    bool operator== (const Point&) const = default;

    friend std::istream& operator>>(std::istream& is, Point& p) {
        return is >> p.x >> p.y;
    }
};

std::vector<Point> getConvexHull(std::vector<Point> points, bool keep_collinear = false) {
    std::ranges::sort(points);
    auto erase_range = std::ranges::unique(points);
    points.erase(erase_range.begin(), erase_range.end());

    int n = ssize(points);
    if (n <= 1) {
        return points;
    }
    std::vector<Point> lower;
    for (int i = 0; i < n; i++) {
        while (ssize(lower) >= 2) {
            auto cross_multi_res = cross_multi(
                lower.back() - lower.at(ssize(lower) - 2),
                points.at(i) - lower.back()
            );
```

```
            if (keep_collinear) {
                if (cross_multi_res < 0) lower.pop_back();
                else break;
            } else {
                if (cross_multi_res <= 0) lower.pop_back();
                else break;
            }
        }

        lower.push_back(points[i]);
    }

    std::vector<Point> upper;
    for (int i = n - 1; i >= 0; i--) {
        while (ssize(upper) >= 2) {
            auto cross_multi_res = cross_multi(
                upper.back() - upper.at(ssize(upper) - 2),
                points.at(i) - upper.back()
            );
            if (keep_collinear) {
                if (cross_multi_res < 0) upper.pop_back();
                else break;
            } else {
                if (cross_multi_res <= 0) upper.pop_back();
                else break;
            }
        }

        upper.push_back(points[i]);
    }

    if(!lower.empty()) lower.pop_back();
    if(!upper.empty()) upper.pop_back();
    std::vector<Point> hull(lower.size() + upper.size());
    std::ranges::copy(lower, hull.begin());
    std::ranges::copy(upper, hull.begin() + lower.size());

    if (hull.empty() && !points.empty()) {
        if (keep_collinear) {
            return points;
        } else {
            std::vector<Point> endpts;
            endpts.push_back(points.front());
            if (!(points.front() == points.back())) endpts.push_back(points.back());
            return endpts;
        }
    }

    return hull;
}
```

DataStructure

Fenwick.hpp

```
#include <bits/stdc++.h>
#include <acm/TypeDef.hpp>

namespace FenwickTree
{

u32 lowbit(u32 x) {
    return x & -x;
}

template <class T>
struct PointUpdateRangeQuery
{
    PointUpdateRangeQuery(const u32 size): tree(size + 1) {}
    PointUpdateRangeQuery(const std::vector<T>& data): tree(data.size() + 1) {
        for (int i = 0; i < std::ssize(data); i++) {
            update(i, data[i]);
        }
    }

    void update(const int idx, const T& val) {
        for (int i = idx + 1; i < std::ssize(tree); i += lowbit(i)) {
            tree[i] += val;
        }
    }

    T query(int idx) const {
        T res{};
        for (int i = idx + 1; i > 0; i -= lowbit(i)) {
            res += tree[i];
        }
        return res;
    }

    T query(int begin, int end) const {
        return query(end - 1) - query(begin - 1);
    }

private:
    std::vector<T> tree;
};

template <class T>
using PURQ = PointUpdateRangeQuery<T>;

template <class T>
struct RangeUpdatePointQuery
{
    RangeUpdatePointQuery(const u32 size): tree(size + 1) {}
```

```
    RangeUpdatePointQuery(const std::vector<T>& data): tree(data.size() + 1) {
        for (int i = 0; i < std::ssize(data); i++) {
            update(i, data[i]);
        }
    }

    void update(const int idx, const T& val) {
        update(idx, idx + 1, val);
    }

    void update(const int begin, const int end, const T& val) {
        for (int i = begin + 1; i <= std::ssize(tree); i += lowbit(i)) {
            tree[i - 1] += val;
        }
        for (int i = end + 1; i <= std::ssize(tree); i += lowbit(i)) {
            tree[i - 1] -= val;
        }
    }

    T query(const int idx) const {
        T res{};
        for (int i = idx + 1; i > 0; i -= lowbit(i)) {
            res += tree[i - 1];
        }
        return res;
    }

private:
    std::vector<T> tree;
};

template <class T>
using RUPQ = RangeUpdatePointQuery<T>;

} // namespace FenwickTree
```

Float.hpp

```
#pragma once

#include <cmath>
#include <iostream>
#include <utility>

template <class T = double, T Eps = 1e-7>
struct FloatWithEps
{
    using value_type = T;
    value_type num_;

    FloatWithEps(): num_() {}
    FloatWithEps(value_type num): num_(std::move(num)) {}
    FloatWithEps(const FloatWithEps&) = default;
```

```
    FloatWithEps(FloatWithEps&&) = default;
    FloatWithEps& operator=(const FloatWithEps&) = default;
    FloatWithEps& operator=(FloatWithEps&&) = default;

    friend FloatWithEps operator+(const FloatWithEps& lhs, const FloatWithEps& rhs) {
        return lhs.num_ + rhs.num_;
    }

    friend FloatWithEps operator-(const FloatWithEps& lhs, const FloatWithEps& rhs) {
        return lhs.num_ - rhs.num_;
    }

    friend FloatWithEps operator*(const FloatWithEps& lhs, const FloatWithEps& rhs) {
        return lhs.num_ * rhs.num_;
    }

    friend FloatWithEps operator/(const FloatWithEps& lhs, const FloatWithEps& rhs) {
        return lhs.num_ / rhs.num_;
    }

    explicit operator bool() const {
        using std::abs;
        return abs(num_) <= Eps;
    }

    bool operator!() const {
        return !bool(*this);
    }

    friend std::partial_ordering operator<=>(const FloatWithEps& lhs, const FloatWithEps&
rhs) {
        auto diff = lhs.num_ - rhs.num_;
        using std::abs;
        if (abs(diff) <= Eps) {
            return std::partial_ordering::equivalent;
        }
        return diff <=> FloatWithEps::value_type(0.0);
    }

    friend bool operator==(const FloatWithEps& lhs, const FloatWithEps& rhs) {
        using std::abs;
        return abs(lhs.num_ - rhs.num_) <= Eps;
    }

    friend bool operator!=(const FloatWithEps& lhs, const FloatWithEps& rhs) {
        return !(lhs == rhs);
    }

    FloatWithEps& operator+=(const FloatWithEps& rhs) {
        num_ += rhs.num_;
        return *this;
    }
```

```
    FloatWithEps& operator-=(const FloatWithEps& rhs) {
        num_ -= rhs.num_;
        return *this;
    }

    FloatWithEps& operator*=(const FloatWithEps& rhs) {
        num_ *= rhs.num_;
        return *this;
    }

    FloatWithEps& operator/=(const FloatWithEps& rhs) {
        num_ /= rhs.num_;
        return *this;
    }

    FloatWithEps& operator++() {
        num_++;
        return *this;
    }

    FloatWithEps operator++(int) {
        FloatWithEps result = *this;
        num_++;
        return result;
    }

    FloatWithEps& operator--() {
        num_--;
        return *this;
    }

    FloatWithEps operator--(int) {
        FloatWithEps result = *this;
        num_--;
        return result;
    }

    friend auto& operator<<(std::ostream& os, const FloatWithEps& num) {
        return os << num.num_;
    }

    friend auto& operator>>(std::istream& is, FloatWithEps& num) {
        return is >> num.num_;
    }

    FloatWithEps abs() const {
        return std::abs(num_);
    }

    friend auto abs(const FloatWithEps& num) {
        return num.abs();
    }
};
```

```
using Float = FloatWithEps<double, 1e-7>;
```

UnionFindSet.hpp

```
#pragma once

#include<vector>

// u32
#include <acm/TypeDef.hpp>

struct UnionFindSet
{
    using UnionIndexType = u32;

    UnionFindSet() = delete;
    explicit UnionFindSet(size_t n):parent(n, 0),rank(n, 0){
        for(size_t i = 0; i < n; i++) {
            parent[i] = i;
        }
    }
    UnionFindSet(const UnionFindSet& ufs) = default;
    UnionFindSet(UnionFindSet&&) noexcept = default;
    UnionFindSet& operator=(const UnionFindSet&) = default;
    UnionFindSet& operator=(UnionFindSet&&) noexcept = default;

    UnionIndexType root(UnionIndexType x) {
        return parent[x] = (parent[x] == x ? x : root(parent[x]));
    }

    void merge(UnionIndexType x, UnionIndexType y) {
        x = root(x), y = root(y);
        if(x == y)
            return;
        if(rank[x] < rank[y])
            parent[x] = y;
        else {
            parent[y] = x;
            if(rank[x] == rank[y]) {
                rank[x]++;
            }
        }
    }

private:
    std::vector<UnionIndexType> parent,rank;
};
```

BigInt.hpp

```cpp
#pragma once
#include <vector>
#include <format>
#include <sstream>
#include <iomanip>
#include <concepts>
// i64 u32
#include <acm/TypeDef.hpp>

template <class QuotType, class RemType>
struct DivModResult
{
    QuotType quot;
    RemType rem;
};

struct BigInt
{
    using number_type = std::vector<int>;
    using sign_type = int;

    inline static constexpr size_t base = 1'000'000'000;
    inline static constexpr size_t base_digits = 9;
    sign_type sign{1};
    number_type number;

    BigInt() : number{} {}
    template <std::integral T>
    BigInt(T num) : sign(num < 0 ? (num = -num, -1) : 1), number{int(num % base), int(num /
base)} {remove_leading_zero();}
    BigInt(const std::string& str) {std::istringstream(str) >> (*this);}
    BigInt(const BigInt& num) = default;
    BigInt(BigInt&& num) noexcept = default;
    BigInt& operator=(const BigInt&) = default;
    BigInt& operator=(BigInt&&) noexcept = default;

    auto operator<=>(const BigInt& other) const {
        if (
            const auto cmp = sign <=> other.sign;
            cmp != 0
        )
            return cmp;
        if (auto cmp = (number.size() <=> other.number.size()); cmp != 0)
            return cmp;
        return std::lexicographical_compare_three_way(
            number.rbegin(), number.rend(),
            other.number.rbegin(), other.number.rend()
        );
    }
    bool operator==(const BigInt& other) const = default;

    friend BigInt abs(const BigInt& num) {
        auto tmp = num;
```

```
        tmp.sign = 1;
        return tmp;
    }

    friend DivModResult<BigInt, BigInt> divmod(
        const BigInt& a,
        const BigInt& b
    ) {
        int norm = BigInt::base / (b.number.back() + 1);
        BigInt
            x = abs(a) * norm,
            y = abs(b) * norm,
            quot, rem;
        quot.number.resize(x.number.size());
        rem.number.reserve(y.number.size() + 1);
        for (int i = std::ssize(x.number) - 1; i >= 0; i--) {
            rem *= BigInt::base;
            rem += x.number[i];
            int
                s1 = (rem.number.size() > y.number.size() ? rem.number[y.number.size()] :
0),
                s2 = (rem.number.size() > y.number.size() - 1 ? rem.number[y.number.size() -
1] : 0);
            int d = (i64(BigInt::base) * s1 + s2) / y.number.back();
            rem -= y * d;
            while (rem < 0)
                rem += b, d--;
            quot.number[i] = d;
        }

        quot.sign = a.sign * b.sign;
        rem.sign = a.sign;
        quot.remove_leading_zero();
        rem.remove_leading_zero();
        return DivModResult<BigInt, BigInt>(quot, rem / norm);
    }

    friend std::istream& operator>>(std::istream& is, BigInt& num) {
        std::string str;
        is >> str;
        num.sign = 1;
        size_t pos = 0;
        while (pos < str.length() && (str[pos] == '-' || str[pos] == '+')) {
            if (str[pos] == '-') num.sign = -1;
            pos++;
        }
        for (int i = int(str.length()) - 1; i >= i64(pos); i -= BigInt::base_digits) {
            u32 tmp = 0;
            for (int j = std::max(i64(pos), i - i64(BigInt::base_digits) + 1); j <= i; j++)
                (tmp *= 10) += (str[j] - '0');
            num.number.push_back(tmp);
        }
        num.remove_leading_zero();
```

```
        return is;
    }

    friend std::ostream& operator<<(std::ostream& os, const BigInt& num) {
        auto& [sign, number] = num;
        if (num.number.empty() == true)
            return os << '0';

        if (num.sign == -1)
            os << '-';
        os << (num.number.empty() ? 0 : num.number.back());
        for (int i = std::ssize(num.number) - 2; i >= 0; i--)
            os << std::setw(BigInt::base_digits) << std::setfill('0') << number[i];

        return os;
    }

    BigInt operator-() const {
        auto res = *this;
        res.sign = -sign;
        return res;
    }

    BigInt operator+(const BigInt& other) const {
        if (sign != other.sign)
            return *this - (-other);

        auto res = BigInt{};
        res.sign = this->sign;
        const auto& [shorter, longer] =
            number.size() < other.number.size() ?
            std::tie(this->number, other.number) :
            std::tie(other.number, this->number);
        res.number.resize(longer.size() + 1);

        number_type& res_num = res.number;
        i64 short_len = shorter.size();
        for (int i = 0; i < short_len; i++)
            res_num[i] = shorter[i] + longer[i];
        std::ranges::copy(longer.begin() + short_len, longer.end(), res_num.begin() +
short_len);
        for (int i = 0, rem = 0; i < std::ssize(res_num) || rem != 0; i++) {
            res_num[i] += rem;
            rem = res_num[i] >= i64(base);
            if (rem)
                res_num[i] -= base;
        }
        res.remove_leading_zero();

        return res;
    }

    BigInt operator-(const BigInt& other) const {
```

```
        if (sign != other.sign)
            return *this + (-other);

        if (abs(*this) < abs(other))
            return -(other - *this);


        const auto& [subed_num, sub_num] = std::tie(number, other.number);

        auto res = BigInt(0);
        res.sign = this->sign;
        auto& res_num = res.number;
        res_num.resize(subed_num.size() + 1);
        auto len = std::ssize(sub_num);;
        for (int i = 0; i < len; i++)
            res_num[i] = subed_num[i] - sub_num[i];

        std::ranges::copy(subed_num.begin() + len, subed_num.end(), res_num.begin() + len);
        for (int i = 0, rem = 0; i < std::ssize(res_num) || rem; i++) {
            res_num[i] -= rem;
            rem = res_num[i] < 0;
            if (rem) {
                res_num[i] += base;
            }
        }

        res.remove_leading_zero();

        return res;
    }

    BigInt operator*(const BigInt& other) const {
        std::vector<i64>
            a6 = convert_base<i64>(number, base_digits, 6),
            b6 = convert_base<i64>(other.number, base_digits, 6);
        const auto max_n = std::bit_ceil(std::ranges::max(a6.size(), b6.size()));
        a6.resize(max_n), b6.resize(max_n);
        auto c = karatsuba(a6, b6);
        BigInt res;
        res.sign = sign * other.sign;
        res.number.reserve((std::ssize(c) * 6 + base_digits - 1) / base_digits);
        for (int i = 0, rem = 0; i < std::ssize(c); i++) {
            i64 cur = c[i] + rem;
            res.number.push_back(int(cur % 1'000'000));
            rem = int(cur / 1'000'000);
        }
        res.number = convert_base<int>(res.number, 6, base_digits);
        res.remove_leading_zero();

        return res;
    }

    BigInt operator/(int num) const {
```

```
        BigInt res = *this;
        res.sign *= (num < 0 ? -1 : 1);
        num *= (num < 0 ? -1 : 1);
        for (int i = std::ssize(res.number) - 1, rem = 0; i >= 0; i--) {
            i64 cur = res.number[i] + rem * i64(base);
            res.number[i] = int(cur / num);
            rem = int(cur % num);
        }
        res.remove_leading_zero();
        return res;
    }

    BigInt& operator/=(int num) {
        return *this = *this / num;
    }

    BigInt operator/(i64 num) const {
        return *this / BigInt(num);
    }

    BigInt& operator/=(i64 num) {
        return *this /= BigInt(num);
    }

    BigInt operator/(const BigInt& other) const {
        return divmod(*this, other).quot;
    }

    BigInt operator%(const BigInt& other) const {
        return divmod(*this, other).rem;
    }

    BigInt& operator+=(const BigInt& num) {
        return *this =  *this + num;
    }

    BigInt& operator-=(const BigInt&& num) {
        return *this = *this - num;
    }

    BigInt& operator*=(const BigInt&& num) {
        return *this = *this * num;
    }

    BigInt& operator/=(const BigInt&& num) {
        return *this = *this / num;
    }

    BigInt& operator%=(const BigInt&& num) {
        return *this = *this % num;
    }

private:
```

```
    void remove_leading_zero() {
        while (number.empty() == false && number.back() == 0)
            number.pop_back();
        sign = number.empty() ? 1 : sign;
    }

    static std::vector<i64> karatsuba(const std::vector<i64>& a, const std::vector<i64>& b)
{
        const auto n = std::ssize(a);
        std::vector<i64> res(n << 1, 0);
        if (n <= 32) {
            for (int i = 0; i < n; i++)
                for (int j = 0; j < n; j++)
                    res[i + j] += a[i] * b[j];
            return res;
        }

        const auto k = n >> 1;
        std::vector<i64>
            a1(a.begin(), a.begin() + k),
            a2(a.begin() + k, a.end()),
            b1(b.begin(), b.begin() + k),
            b2(b.begin() + k, b.end());

        const auto
            a1b1 = karatsuba(a1, b1),
            a2b2 = karatsuba(a2, b2);

        for (int i = 0; i < k; i++)
            a2[i] += a1[i];
        for (int i = 0; i < k; i++)
            b2[i] += b1[i];

        auto c = karatsuba(a2, b2);
        for (int i = 0; i < std::ssize(a1b1); i++)
            c[i] -= a1b1[i];
        for (int i = 0; i < std::ssize(a2b2); i++)
            c[i] -= a2b2[i];

        for (int i = 0; i < std::ssize(a1b1); i++)
            res[i] += a1b1[i];
        for (int i = 0; i < std::ssize(c); i++)
            res[i + k] += c[i];
        for (int i = 0; i < std::ssize(a2b2); i++)
            res[i + n] += a2b2[i];

        return res;
    }

    template<std::integral T, std::integral U>
    static std::vector<T> convert_base(const std::vector<U>& a, const int old_digits, const
int new_digits) {
```

```
        std::vector<u64> pow(new_digits + 1);
        pow[0] = 1;
        for (int i = 1; i <= new_digits; i++)
            pow[i] = pow[i - 1] * 10;

        std::vector<T> res;
        res.reserve((std::ssize(a) * old_digits + new_digits - 1) / new_digits + 1);

        i64 cur = 0;
        int cur_digits = 0;
        for (int i = 0; i < std::ssize(a); i++) {
            cur += a[i] * pow[cur_digits];
            cur_digits += old_digits;
            while (cur_digits >= new_digits) {
                res.push_back((T)(cur % pow[new_digits]));
                cur /= pow[new_digits];
                cur_digits -= new_digits;
            }
        }
        res.push_back((T)(cur));
        while (res.empty() == false && res.back() == 0)
            res.pop_back();

        return res;
    }

};

template <>
struct std::formatter<BigInt> {
    auto parse(std::format_parse_context& ctx) {
        return ctx.begin();
    }

    template <class OutIt>
    auto format(const BigInt& Int, std::basic_format_context<OutIt, char>& ctx) const {
        std::ostringstream oss;
        oss << Int;
        std::string result = oss.str();
        std::ranges::move(result.begin(), result.end(), ctx.out());
        return ctx.out();
    }
};

static_assert(std::formattable<BigInt, char>);
```

ModInt.hpp

```
#pragma once
#include <bits/stdc++.h>

// i64
#include <acm/TypeDef.hpp>
```

```cpp
// fastModPow
#include <acm/Math/FastPow.hpp>

// to_unsigned
#include <acm/type_traits.hpp>

template <u64 MOD = 998'244'353, class T = i64>
struct ModInt
{
    T num;
    constexpr ModInt(): ModInt(T{}) {}
    constexpr ModInt(T x): num(norm(x % MOD)) {}
    constexpr ModInt(const ModInt&) = default;
    constexpr ModInt& operator=(const ModInt&) = default;

    constexpr T norm(T x) const {
        if (x < 0) x += MOD;
        else if (to_unsigned(x) >= MOD) x -= MOD;
        return x;
    }

    constexpr operator T() const {
        return num;
    }

    constexpr ModInt inv() const {
        return fastModPow<MOD, T>(num, MOD - 2);
    }

    friend auto& operator>>(std::istream& is, ModInt& mod_int) {
        is >> mod_int.num;
        mod_int.num %= MOD;
        mod_int.num = mod_int.norm(mod_int.num);
        return is;
    }

    friend auto& operator<<(std::ostream& os, const ModInt& mod_int) {
        return os << mod_int.num;
    }

    constexpr ModInt operator-() const {
        return ModInt(MOD - num);
    }

    constexpr ModInt& operator++() {
        num = norm(num + 1);
        return *this;
    }

    constexpr ModInt operator++(int) {
        auto res = *this;
        ++(*this);
```

```
        return res;
    }

    constexpr ModInt& operator--() {
        num = norm(num - 1);
        return *this;
    }

    constexpr ModInt operator--(int) {
        auto res = *this;
        --(*this);
        return res;
    }

    constexpr ModInt& operator+=(const ModInt& other) {
        num = norm(num + other.num);
        return *this;
    }

    constexpr ModInt& operator-=(const ModInt& other) {
        num = norm(num - other.num);
        return *this;
    }

    constexpr ModInt& operator*=(const ModInt& other) {
        num = i64(num) * other.num % MOD;
        return *this;
    }

    constexpr ModInt& operator/=(const ModInt& other) {
        return *this *= other.inv();
    }

    constexpr ModInt& operator%=(u64 other) {
        return num %= other, *this;
    }

    constexpr ModInt operator+(const ModInt& other) const {return ModInt(num) += other;}

    constexpr ModInt operator-(const ModInt& other) const {return ModInt(num) -= other;}

    constexpr ModInt operator*(const ModInt& other) const {return ModInt(num) *= other;}

    constexpr ModInt operator/(const ModInt& other) const {return ModInt(num) /= other;}

    constexpr ModInt operator%(u64 other) const {return ModInt(num) %= other;}
    auto operator<=>(const ModInt& other) const = default;
    bool operator==(const ModInt& other) const = default;
};

template <u64 MOD, class T>
struct std::formatter<ModInt<MOD, T>> : std::formatter<T> {
    using std::formatter<T>::parse;
```

```
    template <class Out>
    auto format(const ModInt<MOD, T>& Int, std::basic_format_context<Out, char>& ctx) const
{
        return std::formatter<T>::format(Int.num, ctx);
    }
};

/*

template <u64 MOD = 998'244'353, class T = i64>
struct ModInt
{
    #define CE constexpr
    #define C const
    #define CR const ModInt&
    #define OP operator

    T num;
    CE ModInt(): ModInt(T{}) {}
    CE ModInt(T x): num(norm(x % MOD)) {}

    CE ModInt(CR) = default;
    CE ModInt& operator=(CR) = default;
    auto operator<=>(CR) C = default;
    bool operator==(CR) C = default;

    CE T norm(T x) C {
        if (x < 0) x += MOD;
        else if (to_unsigned(x) >= MOD) x -= MOD;
        return x;
    }

    CE explicit operator T() C { return num; }

    CE ModInt inv() C { return fastModPow<MOD, T>(num, MOD - 2); }

    friend auto& operator>>(std::istream& is, ModInt& n) {
        is >> n.num;
        n.num %= MOD;
        n.num = n.norm(n.num);
        return is;
    }

    friend auto& operator<<(std::ostream& os, const ModInt& n) {
        return os << n.num;
    }

    CE ModInt operator-() C {
        return ModInt(MOD - num);
    }

    #define OP1(A) CE ModInt OP##A##A##() { \
```

```
        num = norm(num A 1); \
        return *this; \
    } \
    CE ModInt OP##A##A##(int) { \
        auto res = *this; \
        A##A##(*this); \
        return *this; \
    }

    OP1(+) OP1(-)

    #define OP2(A) CE ModInt& OP##A##=(CR o) { \
        num = norm(num A o.num); \
        return *this; \
    } \
    CE ModInt OP##A(CR o) C { return ModInt(num) A##= o; }

    // + - * / %
    // regex: from "(.) ", to "OP2($1) "
    OP2(+) OP2(-) OP2(*) OP2(/) OP2(%)

    // CE C CR OP OP1 OP2
    // regex: from "(\w+)", to "#undef $1 [shift + enter]"
    #undef CE
    #undef C
    #undef CR
    #undef OP
    #undef OP1
    #undef OP2
};

*/
```

NumberTheory

PrimeSieve.hpp

```
#pragma once
#include <vector>
#include <acm/TypeDef.hpp>

struct EularSieve
{
    std::vector<int> primes;
    std::vector<int> min_factor;

    EularSieve(const i64 N): min_factor(N + 1, 0) {
        min_factor[1] = 1;
        for (int i = 2; i <= N; i++) {
            if (min_factor[i] == 0) {
                min_factor[i] = i;
                primes.push_back(i);
```

```
            }
            for (auto p : primes) {
                if (i64(i) * p > N) {
                    break;
                }
                min_factor[i * p] =  p;
                if (p == min_factor[i]) {
                    break;
                }
            }
        }
    }

    int operator[](int idx) const {
        return primes[idx - 1];
    }

    bool is_prime(int val) const {
        return min_factor[val] == 0;
    }
};
```

TypeDef.hpp

```
#pragma once

#include <acm/int128.hpp>

using i8  = std::int8_t;
using u8  = std::uint8_t;
using i16 = std::int16_t;
using u16 = std::uint16_t;
using i32 = std::int32_t;
using u32 = std::uint32_t;
using i64 = std::int64_t;
using u64 = std::uint64_t;
template <class T>
using ref = T&;
template <class T>
using cref = const T&;
```

type_traits.hpp

```
#pragma once

#include <type_traits>
#include <acm/TypeDef.hpp>

template <class T>
auto to_unsigned(T Int) {
    return std::make_unsigned_t<T>(Int);
```

```
}

u128 to_unsigned(const i128& Int) {
    return u128(Int);
}

u128 to_unsigned(const u128& Int) {
    return Int;
}

template <class T>
auto to_signed(T Int) {
    return std::make_signed_t<T>(Int);
}

i128 to_signed(const i128& Int) {
    return Int;
}

i128 to_signed(const u128& Int) {
    return i128(Int);
}
```

String

Manacher.hpp

```
#pragma once
#include <bits/stdc++.h>

template <char splitChar = '#'>
struct Manacher
{
    std::string str;
    std::vector<int> find(std::string_view s) {
        str.reserve(s.size() << 1 | 1);
        str.push_back(splitChar);
        for (auto ch : s) str.push_back(ch), str.push_back(splitChar);

        auto d = std::vector<int>(str.size());
        for (int i{}, l{}, r{}; i < std::ssize(d); i++) {
            int k = (i > r) ? 1 : std::min(d[l + r - i], r - i + 1);
            while (0 <= i - k && i + k < std::ssize(d) && str[i - k] == str[i + k])
                k++;
            d[i] = k--;
            if (i + k > r) l = i - k, r = i + k;
        }
        return d;
    }
};
```

MinLexRotator.hpp

```
#pragma once
#include <bits/stdc++.h>

// 最小表示法: 对于字符串s,如果s[i..n] + s[1..i-1] = t,则称s与t循环同构,找到与S循环同构的字符串中字典序最小的字符串

template<class T>
size_t minLexRotator(std::string_view arr) {
    size_t i{}, j{1}, k{0};
    size_t n = arr.size();
    while (i < n && j < n && k < n) {
        if (arr[(i + k) % n] == arr[(j + k) % n])
            k++;
        else {
            if (arr[(i + k) % n] > arr[(j + k) % n])
                i += k + 1;
            else
                j += k + 1;
            i += (i == j);
            k = 0;
        }
    }
    return std::min(i, j);
}
```

Trie.hpp

```
#pragma once
#include <bits/stdc++.h>

namespace Trie
{
struct LowerToIndex {
    int operator()(char ch) const {
        return ch - 'a';
    }
};

struct UpperToIndex {
    int operator()(char ch) const {
        return ch - 'A';
    }
};

struct AlphaToIndex {
    int operator()(char ch) const {
        if (std::isupper(ch)) {
            return ch - 'A';
        } else {
            return ch - 'a' + 'A';
```

```
        }
    }
};

struct AlnumToIndex {
    int operator()(char ch) const {
        if (std::isdigit(ch)) {
            return ch - '0';
        } else if (std::isupper(ch)) {
            return ch - 'A' + '0';
        } else {
            return ch - 'a' + '0' + 'A';
        }
    }
};

template <int ASCIISize = 26, class MapFunc = LowerToIndex>
struct ASCII
{
    using Node = std::array<int, ASCIISize>;

    ASCII(int size): tree(size, Node{}), exist(size, 0), cnt(0), mapFunc(MapFunc{}) {}

    void insert(std::string_view s) {
        int p = 0;
        for (auto ch : s) {
            int c = mapFunc(ch);
            if (!tree[p][c]) tree[p][c] = ++cnt;
            p = tree[p][c];
            exist[p]++;
        }
    }

    int search(std::string_view s) const {
        int p = 0;
        for (auto ch : s) {
            int c = mapFunc(ch);
            if (!tree[p][c]) return 0;
            p = tree[p][c];
        }
        return exist[p];
    }

private:
    std::vector<Node> tree;
    std::vector<int> exist;
    int cnt;
    [[no_unique_address]] MapFunc mapFunc;
};

template <int BitSize = 31>
struct MaxXor {
    struct Node {
```

```
        std::array<int, 2> son_idx{};
        int size{};
    };

    MaxXor(int size): tree(size, Node{}), cnt(0) {}

    void insert(int val) {
        int p = 0;
        tree[p].size++;
        for (int i = Bit - 1; i >= 0; i--) {
            int bit = (val >> i) & 1;
            if (!tree[p].son_idx[bit]) tree[p].son_idx[bit] = ++cnt;
            p = tree[p].son_idx[bit];
            tree[p].size++;
        }
    }

    int query(int val) const {
        int p = 0;
        for (int i = Bit - 1; i >= 0; i--) {
            int bit = (val >> i) & 1;
            if (!tree[p].son_idx[bit]) return 0;
            p = tree[p].son_idx[bit];
        }
        return tree[p].size;
    }

    int maxXor(int val) const {
        int res = 0, p = 0;
        for (int i = Bit - 1; i >= 0; i--) {
            int bit = (val >> i) & 1;
            if (tree[p].son_idx[bit ^ 1]) {
                res += 1 << i;
                bit ^= 1;
            }
            p = tree[p].son_idx[bit];
        }
        return res;
    }

private:
    std::vector<Node> tree;
    int cnt{};
};


template <size_t Bit = 31>
struct AllXor
{
    struct Node
    {
        std::array<int, 2> son_idx{};
        int size{};
```

```
        int xor_val{};
    };

    AllXor(size_t size = 200'000) : tree(size * Bit, Node{}), cnt{0} {}

    void xorSon(int idx) {
        tree[idx].size = tree[idx].xor_val = 0;
        if (tree[idx].son_idx[0]) {
            tree[idx].size += tree[tree[idx].son_idx[0]].size;
            tree[idx].xor_val ^= tree[tree[idx].son_idx[0]].xor_val << 1;
        }
        if (tree[idx].son_idx[1]) {
            tree[idx].size += tree[tree[idx].son_idx[1]].size;
            tree[idx].xor_val ^= tree[tree[idx].son_idx[1]].xor_val << 1 |
(tree[tree[idx].son_idx[1]].size & 1);
        }
        tree[idx].size &= 1;
    }

    void insert(int val) {
        std::vector<int> stack;
        stack.reserve(Bit + 1);
        int now = 0;
        stack.push_back(now);
        for (int i = 0; i < (long)(Bit); i++) {
            int bit = (val >> i) & 1;
            if (tree[now].son_idx[bit] == 0) tree[now].son_idx[bit] = ++cnt;
            now = tree[now].son_idx[bit];
            stack.push_back(now);
        }
        tree[stack.back()].size++;
        stack.pop_back();
        while(stack.size()) {
            xorSon(stack.back());
            stack.pop_back();
        }
    }

    int search(int val) const {
        int now = 0;
        for (int i = 0; i < Bit; i++) {
            int bit = (val >> i) & 1;
            if (!tree[now].son_idx[bit]) return 0;
            now = tree[now].son_idx[bit];
        }
        return tree[now].size;
    }

    void erase(int val) {
        if (search(val) == 0)
            return;
        std::vector<int> stack;
        stack.reserve(Bit + 1);
```

```
        int now = 0;
        stack.push_back(now);
        for (int i = 0; i < Bit; i++) {
            int bit = (val >> i) & 1;
            now = tree[now].son_idx[bit];
            stack.push_back(now);
        }
        tree[stack.back()].size--;
        stack.pop_back();
        while (tree.size()) {
            xorSon(stack.back());
            stack.pop_back();
        }
    }

    void add() {
        std::vector<int> stack;
        stack.reserve(Bit + 1);
        int now = 0;
        std::swap(tree[now].son_idx[0], tree[now].son_idx[1]);
        stack.push_back(now);
        while(tree[now].son_idx[0]) {
            now = tree[now].son_idx[0];
            std::swap(tree[now].son_idx[0], tree[now].son_idx[1]);
            stack.push_back(now);
        }
        while(stack.size()) {
            xorSon(stack.back());
            stack.pop_back();
        }
    }

    int allXorValue() const {
        return tree[0].xor_val;
    }
private:
    std::vector<Node> tree;
    int cnt = 0;
};

} // namespace Trie
```

KMP.hpp

```
#include <bits/stdc++.h>
struct KMP
{
    std::string pattern;
    std::vector<int> border;

    KMP() {};
    KMP(auto&& str) : pattern(std::forward<decltype(str)>(str)) {
        init_border();
```

```
    }
    void init(auto&& str) {
        init_pattern(std::forward<decltype(str)>(str));
        init_border();
    }
    void init_border() {
        border.assign(pattern.size(), 0);
        for (int i = 1, j = 0; i < std::ssize(pattern); ++i)
        {
            while (j > 0 && pattern[i] != pattern[j])
            {
                j = border[j - 1];
            }
            if (pattern[i] == pattern[j])j++;
            border[i] = j;
        }
    }
    void init_pattern(auto&& str)
    {
        this->pattern = std::forward<decltype(str)>(str);
    }
    std::vector<int> find(const std::string& text)
    {
        std::vector<int> pos;
        for (int i = 0, j = 0; i < std::ssize(text); ++i)
        {
            while (j > 0 && text[i] != pattern[j])
            {
                j = border[j - 1];
            }
            if (text[i] == pattern[j])j++;
            if (j == std::ssize(pattern))
            {
                pos.push_back(i - j + 1);
                j = border[j - 1];
            }
        }
        return pos;
    }
    auto operator[](int idx) const
    {
        return border[idx];
    }
};
```

ExKMP.hpp

```
#pragma once
#include <bits/stdc++.h>

std::vector<int> exKMP(std::string_view s) {
    std::vector<int> z(s.size(), 0);
    int n = s.size();
```

```
    for (int i{1}, l{}, r{}; i < n; i++) {
        if (i > r)
            z[i] = 0;
        else
            z[i] = std::min(z[i - l], r - i + 1);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;
        if (i + z[i] - 1 > r)
            l = i, r = i + z[i] - 1;
    }
    return z;
}
```

Lyndon.hpp

```
#pragma once
#include <bits/stdc++.h>

// Lydon串: 字符串 s 的字典序严格小于s的所有后缀
// Lydon分解: 字符串 s = w_1 w_2 w_3 .. w_k,其中w_i为Lydon串,且字典序w_i >= w_i+1,这样的分解成为
Lydon分解,这样的分解存在且唯一
std::vector<std::string> lyndon(std::string_view s)  {
    int n = s.size(), i = 0;
    std::vector<std::string> factorization;
    while (i < n) {
    int j = i + 1, k = i;
    while (j < n && s[k] <= s[j]) {
        if (s[k] < s[j])
        k = i;
        else
        k++;
        j++;
    }
    while (i <= k) {
        factorization.emplace_back(s.substr(i, j - k));
        i += j - k;
    }
    }
    return factorization;
}
```

Hash.hpp

```
#pragma once
#include <acm/TypeDef.h>
#include <bits/stdc++.h>
#include <acm/DataStructure/ModInt.hpp>

template <i64 Base, class T = i64>
struct PowerBase
{
    inline static std::vector<T> powerBase = std::vector<T>();
```

```
    PowerBase(size_t n) {
        Init(n);
    }
    PowerBase& Init(size_t n) {
        powerBase.resize(n + 1);
        powerBase[0] = 1;
        for (
            int i = 1;
            i <= i64(n);
            i++
        ) {
            powerBase[i] = powerBase[i - 1] * Base;
        }
        return *this;
    }
};

template <i64 Mod, i64 Base>
struct Hash : PowerBase<Base, ModInt<Mod>>
{
    std::vector<ModInt<Mod>> hash;
    Hash(const std::string& s) : PowerBase<Base, ModInt<Mod>>(s.size()), hash(1 + s.size())
{
        for (int i = 1; i <= std::ssize(s); i++) {
            hash[i] = hash[i - 1] * Base + s[i - 1];
        }
    }
    i64 get(int l, int r) const {
        return i64(hash[r + 1] - hash[l] * PowerBase<Base, ModInt<Mod>>::powerBase[r - l +
1]);
    }
};

template <
    i64 Mod1 = 1'000'000'007ll, i64 Base1 = 131ll,
    i64 Mod2 = 1'000'000'009ll, i64 Base2 = 13331l
>
struct DoubleHash
{
    Hash<Mod1, Base1> hash_1;
    Hash<Mod2, Base2> hash_2;
    DoubleHash(const std::string& s) : hash_1(s), hash_2(s) {}
    std::pair<i64, i64> get(int l, int r) {
        return {hash_1.get(l, r), hash_2.get(l, r)};
    }
};
```

int128.hpp

```
#pragma once

#include <iostream>
#include <string>
```

```
#include <algorithm>

using i128 = __int128_t;
using u128 = __uint128_t;

std::istream& operator>>(std::istream& is, i128& n) {
    std::string s;
    if (!(is >> s)) return is;

    n = 0;
    std::size_t i = 0;
    bool negetive = false;

    if (s[0] == '-') {
        negetive = true;
        i = 1;
    } else if (s[0] == '+') {
        i = 1;
    }

    for (; i < s.size(); i++) {
        if (isdigit(s[i])) {
            n = n * 10 + (s[i] - '0');
        } else [[unlikely]] {
            is.setstate(std::ios::failbit);
            break;
        }
    }

    if (negetive) n = -n;
    return is;
}

std::istream& operator>>(std::istream& is, u128& n) {
    std::string s;
    if (!(is >> s)) return is;

    n = 0;
    std::size_t i = 0;

    if (s[0] == '-') {
        is.setstate(std::ios::failbit);
        return is;
    } else if (s[0] == '+') {
        i = 1;
    }

    for (; i < s.size(); i++) {
        if (isdigit(s[i])) {
            n = n * 10 + (s[i] - '0');
        } else [[unlikely]] {
            is.setstate(std::ios::failbit);
            break;
```

```
        }
    }

    return is;
}

std::ostream& operator<<(std::ostream& os, i128 n) {
    if (n == 0) return os << "0";
    if (n < 0) {
        os << "-";
        n = -n;
    }
    std::string s;
    while (n > 0) {
        s += (char)('0' + (n % 10));
        n /= 10;
    }
    std::reverse(s.begin(), s.end());
    return os << s;
}

std::ostream& operator<<(std::ostream& os, u128 n) {
    if (n == 0) return os << "0";
    std::string s;
    while (n > 0) {
        s += (char)('0' + (n % 10));
        n /= 10;
    }
    std::reverse(s.begin(), s.end());
    return os << s;
}
```

Math

GaussElim.hpp

```
#pragma once

#include <vector>
#include <cmath>
#include <utility>

#include <acm/DataStructure/Float.hpp>

template <class T = Float>
struct GaussElimSolver {
    using value_type = T;

    enum Solution {None, Infinte, Unique};

    struct Result {
        Solution sol_case;
```

```
        std::vector<value_type> solution;
    };

    Result operator()(std::vector<std::vector<value_type>> A, std::vector<value_type> b)
const {
        auto matrix_size = A.size();
        assert(matrix_size != 0 && b.size() == matrix_size && "Invalid matrix dimensions");
        for (auto& row : A) {
            assert(row.size() == matrix_size && "Invalid matrix dimensions");
        }

        int n = matrix_size;

        for (int i = 0; i < n; i++) {
            int max_value_row = i;
            for (int j = 0; j < n; j++) {
                if (j < i && A[j][j] != 0) {
                    continue;
                }
                using std::abs;
                if (abs(A[j][i]) > abs(A[max_value_row][i])) {
                    max_value_row = j;
                }
            }
            std::swap(A[i], A[max_value_row]);
            std::swap(b[i], b[max_value_row]);
            max_value_row = i;
            if (A[max_value_row][i] != 0) {
                auto tmp = A[max_value_row][i];
                for (int j = i; j < n; j++) {
                    A[max_value_row][j] /= tmp;
                }
                b[max_value_row] /= tmp;

                for (int j = 0; j < n; j++) {
                    if (i == j) continue;
                    auto rate = A[j][i] / A[max_value_row][i];
                    for (int k = 0; k < n; k++) {
                        A[j][k] -= A[max_value_row][k] * rate;
                    }
                    b[j] -= b[max_value_row] * rate;
                }
            }
        }

        Result result;
        result.sol_case = Unique;

        for (int i = 0; i < n; i++) {
            if (A[i][i] == 0) {
                if (b[i] == 0) {
                    result.sol_case = Infinte;
                    break;
```

```
                } else {
                    result.sol_case = None;
                    break;
                }
            }
        }
        result.solution = std::move(b);
        return result;
    }
};

using GaussElim = GaussElimSolver<>;

constexpr GaussElim gauss_elim;
```

MathAlgorithm.hpp

```
#pragma once
#include <bits/stdc++.h>
template<std::integral T>
T sqrt(T n) {
    double approx = std::sqrt(n);
    T floor_val = static_cast<T>(approx);

    while((floor_val + 1) * (floor_val + 1) <= n) ++floor_val;
    while(floor_val * floor_val > n) --floor_val;

    return floor_val;
}
```

ExGCD.hpp

```
#pragma once
#include <tuple>

template <class T>
struct ExgcdResult {
    T gcd{}, x{}, y{};
};

template <class T>
ExgcdResult<T> exgcd(T a, T b) {
    if (b == 0) {
        return ExgcdResult<T>{a, 1, 0};
    }
    auto [d, x, y] = exgcd(b, a % b);
    return ExgcdResult<T>{d, y, x - (a / b) * y};
};
```

FastPow.hpp

```cpp
#pragma once

#include <acm/TypeDef.hpp>

template <class T>
T fastPow(T base, u64 power) {
    T res = 1;
    for (; power; power >>= 1, base *= base) {
        if (power & 1) {
            res *= base;
        }
    }
    return res;
}

template <u64 MOD = 998'244'353, class T>
T fastModPow(T base, u64 power) {
    T res = 1;
    for (base %= MOD; power; power >>= 1, base = base * base % MOD) {
        if (power & 1) {
            res = res * base % MOD;
        }
    }
    return res;
}

template <class T>
T fastModPow(T base, u64 power, u64 MOD) {
    T res = 1;
    for (base %= MOD; power; power >>= 1, base = base * base % MOD) {
        if (power & 1) {
            res = res * base % MOD;
        }
    }
    return res;
}
```